1. Introduction

This project focuses on designing a relational database for analyzing high-volume Futures and Options (F&O) trading data from multiple exchanges such as NSE, BSE, and MCX.
The dataset used contains information about instruments, expiry dates, strike prices, prices, volume, open interest, and timestamps.The goal of the database design is to support efficient storage, querying, and analysis of trading data while maintaining scalability and performance for large datasets.

2. Database Design

The database schema was designed using **Third Normal Form (3NF)** to reduce redundancy and maintain data integrity. The design separates the dataset into logical entities such as:

I).Instruments

This table stores unique financial instruments such as NIFTY, BANKNIFTY, or commodity futures.
Columns include:

- instrument_id (Primary Key)
- symbol
- instrument_type

Separating instruments avoids repeated storage of symbol information in every trade record.

II).Expiries

This table stores information about expiry dates and strike prices for options contracts.
Columns include:

- expiry_id (Primary Key)
- expiry_dt
- strike_pr
- option_typ

Separating expiry information improves efficiency when analyzing option chains.

III).Exchanges

This table stores the exchange information where the trade is executed.

Columns include:
exchange_id (Primary Key)

- exchange_name (NSE, BSE, MCX)

This allows the schema to support **multi-exchange trading data**.

IV).Trades

This table stores the actual trading data including OHLC prices, volume, open interest, and timestamps.
Columns include:

- trade_id (Primary Key)
- instrument_id (Foreign Key)
- expiry_id (Foreign Key)
- exchange_id (Foreign Key)
- timestamp
- open_pr
- high_pr
- low_pr
- close_pr
- volume
- open_int

This table is designed to handle large volumes of time-series trading data.

2. Query Design

Several SQL queries were implemented to analyze trading patterns.
Examples include:

- Top 10 symbols by open interest
- 7-day rolling volatility calculation using window functions
- Option chain volume summary grouped by expiry and strike price
- Maximum trading volume analysis
- Cross-exchange price comparisons

These queries demonstrate the ability to perform **financial market analytics using SQL**.

4. Performance Optimization

To improve query performance, indexing strategies were considered.
Indexes can be created on the following columns:

- symbol
- timestamp
- exchange

These indexes help speed up time-series queries and filtering operations.
Additionally, query optimization techniques such as **window functions and aggregation functions** were used to efficiently process trading data.
The **EXPLAIN query plan** was used to analyze how the database executes queries and to ensure efficient execution.

5. Conclusion

The proposed database schema provides a scalable and efficient structure for storing and analyzing Futures and Options trading data.
By using normalization, indexing strategies, and optimized SQL queries, the system supports advanced financial analytics and can easily scale for larger datasets and additional exchanges.